

Next.js Gerçek Kullanım

API, Error Handling, Auth Flow

Session Akışı

Süre	Konu	Anahtar Kavramlar
0-25 dk	API Entegrasyonu & Loading	Server vs Client, "use client", Suspense
25-50 dk	Error Handling & UI Feedback	error.tsx, try/catch, toast
50-70 dk	Auth Flow & Protected Route	cookie, middleware, session
70-90 dk	Pratik: Login + Protected	form action, cookie, redirect

1 ⇄ API Entegrasyonu ve Loading State

🕒 0–25 dakika

1.1 Server Component vs Client Component

Session 6’da Next.js’in varsayılan olarak **server component** kullandığını gördük. Ama gerçek projelerde `useState`, `onClick` gibi interaktif şeylere ihtiyacımız var. Bunlar sadece **client component**’lerde çalışır.

Server Component (varsayılan)

- ✓ `async/await` ile veri çekme
- ✓ DB / API key güvenli
- ✓ JS bundle’a eklenmez
- ✗ `useState`, `useEffect` yok
- ✗ `onClick`, `onChange` yok
- ✗ Browser API’leri yok

Client Component ("use client")

- ✓ `useState`, `useEffect` var
- ✓ Event handler’lar var
- ✓ Browser API’leri
- ✗ JS bundle’a eklenir
- ✗ Doğrudan DB erişimi yok
- ✗ Async olamaz

"use client" Directive

Bir dosyanın **en üstüne** `"use client";` yazarsanız o dosya ve içinden import ettikleri **client component** olur. `useState`, `onClick`, `useEffect` kullanabilirsiniz.

Client Component Örneği

```
"use client"; // <-- En üst satır

import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Artır
      </button>
    </div>
  );
}
```

1.2 Hangi Component'i Ne Zaman Seçmeli?

Senaryo	Hangisi?
Sayfa içeriği veri çekiyor (statik gösterim)	Server
Buton, input, form gibi etkileşim var	Client
Search bar, filtre, modal	Client
SEO için önemli statik içerik	Server
localStorage, window kullanılıyor	Client
API key kullanılıyor	Server (zorunlu!)

💡 Karma Yaklaşım — En Yaygın

Sayfanın **kabuğu server component**, **etkileşimli kısımları** küçük client component'lere ayrılır. Bu sayede JS bundle minimum, performans maksimum olur.

Server component'ler, içinde client component'leri render edebilir. Tersisi mümkün değildir (client içinden server import edilemez).

1.2.1 Karma Yaklaşım Örneği

src/app/products/page.tsx (Server)

```
import AddToCartButton from "../AddToCartButton";

interface Product {
  id: number;
  title: string;
  price: number;
}

export default async function ProductsPage() {
  const res = await fetch("https://fakestoreapi.com/products");
  ;
  const products: Product[] = await res.json();

  return (
    <div>
      <h1>Urunler</h1>
      {products.map(p => (
        <div key={p.id}>
          <h2>{p.title}</h2>
          <p>{p.price} TL</p>
          <AddToCartButton productId={p.id} />
        </div>
      ))}
    </div>
  );
}
```

src/app/products/AddToCartButton.tsx (Client)

```
"use client";

import { useState } from "react";

export default function AddToCartButton({ productId }: {
  productId: number }) {
  const [added, setAdded] = useState(false);

  return (
    <button onClick={() => setAdded(true)}>
      {added ? "Eklendi" : "Sepete Ekle"}
    </button>
  );
}
```

1.3 Loading State Yönetimi

1.3.1 Server Component'te — loading.tsx

Server component'te veri `await` ile çekiliyor ve sayfa hazır olunca render ediliyor. Bu sırada `loading.tsx` otomatik gösterilir.

1. Kullanıcı tıklar → 2. loading.tsx gösterilir → 3. Sunucuda veri → 4. Sayfa hazır

src/app/products/loading.tsx

```
// Sayfanın verisi yüklenirken otomatik gösterilir
export default function Loading() {
  return (
    <div style={{ padding: 40 }}>
      <p>Urunler yükleniyor...</p>
      {/* Skeleton UI da konabilir --- icerik genisliginde gri kutular */}
    </div>
  );
}
```

1.3.2 Client Component'te — Manual Loading State

Client tarafında veri çekiyorsak loading'i kendimiz yönetiriz:

Client'ta Loading

```
"use client";

import { useState, useEffect } from "react";

export default function UserList() {
  const [users, setUsers] = useState<User[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("/api/users")
      .then(r => r.json())
      .then(data => {
        setUsers(data);
        setLoading(false);
      });
  }, []);

  if (loading) return <p>Yükleniyor...</p>;

  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

💡 Hangi Yöntemi Seçmeli?

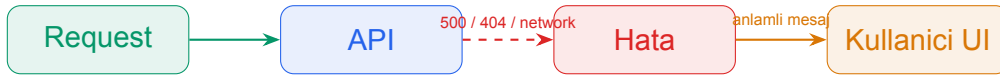
- Sayfa açılırken **veri zaten gerekiyorsa** → Server component + `loading.tsx`
- Veri **kullanıcı etkileşimi** ile gelecekse → Client component + manuel state
- Veri **filtreleme** / arama ile geliyorsa → Client component

2 ⚠️ Error Handling ve UI Feedback

🕒 25–50 dakika

2.1 Hataların Yaşam Döngüsü

API çağrıları her zaman başarılı olmaz. Network kopabilir, sunucu 500 dönebilir, bulunamayan kaynak için 404 gelebilir. Bunları **yakalayıp** kullanıcıya anlamlı bir mesaj göstermeliyiz.



⚠️ Yapılmaması Gerekenler

- Sessizce `console.log(err)` ile bırakmak — kullanıcı bir şey olduğunu bilmez
- Ham hata mesajını göstermek — `TypeError: undefined is not a function` kullanıcıya bir şey anlatmaz
- Tüm sayfa için tek bir “Bir hata oluştu” — nedeni bilinmez
- Beyaz ekran — “site mi çöktü?” sorusu doğar

2.2 Server Component'te Hata Yakalama

2.2.1 Yöntem 1 — try / catch

try / catch ile Hata Yönetimi

```
export default async function ProductsPage() {
  try {
    const res = await fetch("https://api.example.com/products")
    ;

    if (!res.ok) {
      throw new Error(`HTTP ${res.status}: ${res.statusText}`)
      ;
    }

    const products = await res.json();

    return (
      <ul>
        {products.map(p => <li key={p.id}>{p.title}</li>)}
      </ul>
    );
  } catch (err) {
    return (
      <div style={{ padding: 24, color: "#dc2626" }}>
        <h2>Urunler yuklenemedi</h2>
        <p>Lutfen daha sonra tekrar deneyin.</p>
      </div>
    );
  }
}
```


2.2.2 Yöntem 2 — notFound() ile 404'e Yönlendirme

Bulunamayan Kaynak için notFound()

```
import { notFound } from "next/navigation";

export default async function ProductDetailPage({ params }) {
  const { id } = await params;

  const res = await fetch(`https://api.example.com/products/${id}`);

  if (res.status === 404) {
    notFound(); // -> not-found.tsx gösterilir
  }

  if (!res.ok) {
    throw new Error("API hatası"); // -> error.tsx gösterilir
  }

  const product = await res.json();
  return <h1>{product.title}</h1>;
}
```

2.3 error.tsx — Otomatik Error Boundary

error.tsx Nedir?

Next.js'in özel dosyalarından biri. Bir sayfada veya layout'unda **yakalanmamış hata** oluşursa, en yakın `error.tsx` otomatik gösterilir. React'taki Error Boundary'nin Next.js versiyonudur.

src/app/products/error.tsx

```
"use client"; // error.tsx HER ZAMAN client component olmalı

interface Props {
  error: Error & { digest?: string };
  reset: () => void;
}

export default function Error({ error, reset }: Props) {
  return (
    <div style={{ padding: 40, textAlign: "center" }}>
      <h2>Bir seyler ters gitti</h2>
      <p style={{ color: "#64748b" }}>{error.message}</p>
      <button
        onClick={reset}
        style={{
          padding: "10px 20px",
          background: "#2563eb",
          color: "white",
          border: "none",
          borderRadius: 8,
          cursor: "pointer",
        }}
      >
        Tekrar Dene
      </button>
    </div>
  );
}
```

Prop	Açıklama
<code>error</code>	Yakalanan hata objesi (message, digest...)
<code>reset</code>	Çağrılınca segment yeniden render edilir — “Tekrar Dene” butonu için

Error Hiyerarşisi

Bir sayfada hata olduğunda Next.js **en yakın** `error.tsx`'i arar: `products/[id]/error.tsx` → `products/error.tsx` → `global-error.tsx`

Her segment kendi `error.tsx`'ini tanımlayarak farklı UI gösterebilir.

2.4 Client Component'te Hata Yönetimi

Client tarafında veri çekiyorsak, hatayı **state**'te tutarız. Session 4'te öğrendiğimiz **Loading / Error / Data** kalıbı:

Client'ta Loading + Error + Data

```
"use client";

import { useState, useEffect } from "react";

export default function UserList() {
  const [data, setData] = useState<User[] | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    async function load() {
      try {
        const res = await fetch("/api/users");
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const json = await res.json();
        setData(json);
      } catch (err) {
        setError(err instanceof Error ? err.message : "Bilinmeyen hata");
      } finally {
        setLoading(false);
      }
    }
    load();
  }, []);

  if (loading) return <p>Yukleniyor...</p>;
  if (error) return <p style={{ color: "red" }}>Hata: {error}</p>;
  if (!data) return <p>Veri bulunamadi.</p>;

  return <ul>{data.map(u => <li key={u.id}>{u.name}</li>)}</ul>>;
}
```

2.5 UI Feedback Pattern'leri

Pattern	Ne Zaman Kullanılır?
Inline mesaj	Form input altında hata mesajı (Session 4'te gördük)
Toast / Snackbar	Geçici bildirim — “Kaydedildi”, “Silindi” (3-5 sn görünür)
Loading spinner	Buton üzerinde işlem sırasında (<code>disabled</code> + animasyon)
Empty state	Liste boşsa anlamlı mesaj — “Henüz görev yok, ilk ekleyen siz olun!”
Error page	Tüm sayfa çöktüyse <code>error.tsx</code> ile genel hata sayfası
Optimistic UI	İşlem bitmeden UI'ı önden güncelle, hata olursa geri al

💡 Kullanıcı Dostu Mesaj Yazma

- “Bir hata oluştu” yerine: “Ürünler şu an yüklenemiyor, birazdan tekrar deneyin.”
- “500 Internal Server Error” yerine: “Sunucumuzda geçici bir sorun var.”
- Mümkünse **ne yapacağını söyleyin**: “Tekrar dene” butonu, “destek ekibine yazın” linki vs.

3 Auth Flow ve Protected Route

 50–70 dakika

3.1 Authentication Nedir?

Auth Temelleri

Authentication (kimlik doğrulama): “Kim olduğunu kanıtla” (login, email + şifre).

Authorization (yetkilendirme): “Bu işlemi yapmaya yetkin var mı?” (admin sayfasını görebilir mi?).

3.1.1 Tipik Login Akışı



Yöntem	Açıklama
Cookie (httpOnly)	Sunucu set eder, browser otomatik gönderir. En güvenli — JavaScript erişemez, XSS koruması var.
JWT (localStorage)	Token'ı client saklar, header'da gönderir. XSS riski var.
Session (sunucu)	Sunucuda session store; cookie sadece session ID tutar.

Bu Kursta Yaklaşım

En basit ve güvenli olan **httpOnly cookie** yaklaşımını kullanacağız. Gerçek projede arkadaki sistem genelde JWT veya session olsa da, **Next.js tarafından bakınca tek mesele cookie'nin var olup olmadığıdır.**

3.2 Next.js'te Cookie Okuma

Server component'te `cookies()` fonksiyonu ile cookie'leri okuyabiliriz:

Server Component'te Cookie

```
import { cookies } from "next/headers";

export default async function DashboardPage() {
  const cookieStore = await cookies();
  const session = cookieStore.get("session");

  if (!session) {
    return <p>Lutfen giris yapin.</p>;
  }

  return <h1>Hosgeldin, {session.value}!</h1>;
}
```

3.3 Cookie Set Etme — Server Action

Form gönderildiğinde sunucuda kontrol yapıp cookie set etmek için **server action** kullanırız:

src/app/login/actions.ts

```
"use server"; // <-- Server action directive

import { cookies } from "next/headers";
import { redirect } from "next/navigation";

export async function login(formData: FormData) {
  const email = formData.get("email") as string;
  const password = formData.get("password") as string;

  // Gercek projede DB'den kontrol edilir
  if (email === "admin@example.com" && password === "123456")
  {
    const cookieStore = await cookies();
    cookieStore.set("session", email, {
      httpOnly: true,
      secure: true,
      maxAge: 60 * 60 * 24, // 1 gun
      path: "/",
    });
    redirect("/dashboard");
  }

  // Hata durumu: form sayfasina hata mesajı ile geri don
  return { error: "Email veya sifre yanlis" };
}
```

Server Action

"use server" ile işaretlenmiş fonksiyonlar, sunucuda çalışan fonksiyonlardır. Form'un action prop'una doğrudan verilebilir. Browser otomatik olarak POST request gönderir, sonuç sayfaya döner.

3.4 Protected Route — Layout ile Koruma

Bir route grubunu korumak için layout'ta `cookies()` ile kontrol yapıp gerekirse `redirect` kullanırız:

src/app/dashboard/layout.tsx

```
import { cookies } from "next/headers";
import { redirect } from "next/navigation";

export default async function DashboardLayout({
  children,
}): {
  children: React.ReactNode;
}) {
  const cookieStore = await cookies();
  const session = cookieStore.get("session");

  if (!session) {
    redirect("/login"); // <-- Cookie yoksa login'e yonlendir
  }

  return (
    <div>
      <header>
        <p>Giris yapan: {session.value}</p>
      </header>
      {children}
    </div>
  );
}
```

 Korumalı Layout'un Gücü

dashboard/layout.tsx'te kontrol yaptığımız için **dashboard** altındaki **tüm sayfalar** otomatik korunur: /dashboard, /dashboard/settings, /dashboard/profile...Her sayfa için tek tek kontrol yazmaya gerek yok.

3.5 Middleware — Daha Gelişmiş Koruma

`middleware.ts`, her istek **sayfaya gelmeden önce** çalışan bir fonksiyondur. Auth kontrolü için en güçlü yer:

`src/middleware.ts` (proje köküne yakın)

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
  const session = request.cookies.get("session");

  if (!session) {
    // Cookie yoksa login sayfasına yönlendir
    return NextResponse.redirect(new URL("/login", request.url));
  }

  return NextResponse.next(); // Devam et
}

// Hangi route'larda çalışacağını belirt
export const config = {
  matcher: ["/dashboard/:path*", "/profile/:path*"],
};
```

	Layout-based	Middleware-based
Konumu	Her klasörün <code>layout.tsx</code> 'i	Tek dosya (<code>middleware.ts</code>)
Çalışma zamanı	Sayfa render edilirken	İstek geldiğinde, render'dan önce
Performans	Sayfaya kadar gelir, sonra redirect	Erken redirect, daha hızlı
Karmaşık logic	Component'lerle birlikte	Sadeleştirilmiş — Edge runtime
Önerilen	Basit uygulamalar	Çoklu korumalı route'lar

4 Pratik: Login + Protected Page

 70–90 dakika

☰ Pratik Hedef

Basit bir login sistemi ile korumalı sayfa örneği yapalım:

1. /login — Email + şifre formu (client component, hata gösterimi)
2. Form gönderilince server action ile cookie set
3. /dashboard — Korumalı sayfa (cookie yoksa /login'e yönlendirir)
4. /dashboard/profile — Aynı korumayı miras alır (nested layout)
5. Logout butonu — cookie siler, /login'e yönlendirir

4.1 Dosya Yapısı

```
src/app/
├── layout.tsx
├── page.tsx
├── login/
│   ├── page.tsx
│   ├── actions.ts
│   └── LoginForm.tsx
├── dashboard/
│   ├── layout.tsx
│   ├── page.tsx
│   ├── LogoutButton.tsx
│   └── profile/
│       └── page.tsx
```

4.2 Adım 1 — Server Action

src/app/login/actions.ts

```
"use server";

import { cookies } from "next/headers";
import { redirect } from "next/navigation";

export async function login(_prevState: unknown, formData:
  FormData) {
  const email = formData.get("email") as string;
  const password = formData.get("password") as string;

  // Demo amacli sabit kontrol --- gercek projede DB
  if (email !== "admin@example.com" || password !== "123456")
  {
    return { error: "Email veya sifre yanlis." };
  }

  const cookieStore = await cookies();
  cookieStore.set("session", email, {
    httpOnly: true,
    maxAge: 60 * 60 * 24,
    path: "/",
  });

  redirect("/dashboard");
}

export async function logout() {
  const cookieStore = await cookies();
  cookieStore.delete("session");
  redirect("/login");
}
```


4.3 Adım 2 — Login Formu

src/app/login/LoginForm.tsx (Client)

```
"use client";

import { useActionState } from "react";
import { login } from "../actions";

export default function LoginForm() {
  const [state, formAction, pending] = useActionState(login, null);

  return (
    <form action={formAction} style={{ maxWidth: 320, margin: "0 auto" }}>
      <h1>Giris Yap</h1>

      <input
        name="email"
        type="email"
        placeholder="Email"
        required
        defaultValue="admin@example.com"
        style={{ width: "100%", padding: 10, marginBottom: 8 }}
      />

      <input
        name="password"
        type="password"
        placeholder="Sifre"
        required
        style={{ width: "100%", padding: 10, marginBottom: 8 }}
      />

      {state?.error && (
        <p style={{ color: "#dc2626", fontSize: 14 }}>
          {state.error}
        </p>
      )}

      <button
        type="submit"
        disabled={pending}
        style={{
          width: "100%", padding: 12,
          background: "#2563eb", color: "white",
          border: "none", borderRadius: 8,
          cursor: pending ? "not-allowed" : "pointer",
        }}
      >
        {pending ? "Giris yapiliyor..." : "Giris Yap"}
      </button>
    </form>
  );
}
```

4.4 Adım 3 — Login Sayfası

src/app/login/page.tsx (Server)

```
import LoginForm from "../LoginForm";

export default function LoginPage() {
  return <LoginForm />;
}
```

4.5 Adım 4 — Dashboard Layout (Korumalı)

src/app/dashboard/layout.tsx

```
import { cookies } from "next/headers";
import { redirect } from "next/navigation";
import Link from "next/link";
import LogoutButton from "../LogoutButton";

export default async function DashboardLayout({
  children,
}): {
  children: React.ReactNode;
}) {
  const cookieStore = await cookies();
  const session = cookieStore.get("session");

  if (!session) {
    redirect("/login");
  }

  return (
    <div>
      <header style={{
        padding: 16, background: "#2563eb", color: "white",
        display: "flex", justifyContent: "space-between",
      }}>
        <div>
          <Link href="/dashboard" style={{ color: "white",
            marginRight: 16 }}>
            Dashboard
          </Link>
          <Link href="/dashboard/profile" style={{ color: "
            white" }}>
            Profil
          </Link>
        </div>
        <div>
          <span style={{ marginRight: 12 }}>{session.value}</
            span>
          <LogoutButton />
        </div>
      </header>

      <main style={{ padding: 24 }}>{children}</main>
    </div>
  );
}
```


4.6 Adım 5 — Logout Button

src/app/dashboard/LogoutButton.tsx

```
import { logout } from "../../login/actions";

export default function LogoutButton() {
  return (
    <form action={logout} style={{ display: "inline" }}>
      <button
        type="submit"
        style={{
          background: "transparent",
          color: "white",
          border: "1px solid white",
          padding: "4px 12px",
          borderRadius: 4,
          cursor: "pointer",
        }}
      >
        Cikis
      </button>
    </form>
  );
}
```

4.7 Adım 6 — Dashboard Sayfaları

src/app/dashboard/page.tsx

```
import { cookies } from "next/headers";

export default async function DashboardPage() {
  const cookieStore = await cookies();
  const email = cookieStore.get("session")?.value;

  return (
    <div>
      <h1>Hosgeldin!</h1>
      <p>Giris yapan kullanıcı: <strong>{email}</strong></p>
      <p>Bu sayfa sadece giris yapanlar gorebilir.</p>
    </div>
  );
}
```

src/app/dashboard/profile/page.tsx

```
import { cookies } from "next/headers";

export default async function ProfilePage() {
  const cookieStore = await cookies();
  const email = cookieStore.get("session")?.value;

  return (
    <div>
      <h1>Profil</h1>
      <ul>
        <li>Email: {email}</li>
        <li>Rol: Admin</li>
        <li>Uyelik: 2026</li>
      </ul>
    </div>
  );
}
```

💡 Bu Uygulamada Neler Yaptık?

- Server action ile sunucuda form işleme ("use server")
- useActionState ile form state ve loading yönetimi
- httpOnly cookie ile session saklama
- Korumalı layout ile alt sayfaları otomatik korunma
- Server component'te cookies() ile veriye erişim
- Logout için cookie silme

5 Özet ve Sonraki Session

5.1 Bu Session'da Öğrendiklerimiz

- ✓ Server component vs Client component
- ✓ "use client" ne zaman ve nasıl
- ✓ Karma yaklaşım (server + küçük client'lar)
- ✓ `loading.tsx` ile otomatik loading UI
- ✓ Client'ta manuel loading/error state yönetimi
- ✓ Server'da `try/catch` ile hata yakalama
- ✓ `notFound()` vs `throw Error`
- ✓ `error.tsx` ile otomatik error boundary
- ✓ UI feedback pattern'leri (toast, empty state, optimistic)
- ✓ Auth temelleri: cookie, JWT, session
- ✓ Server action ile form işleme ("use server")
- ✓ `useActionState` ile form yönetimi
- ✓ Protected route: layout-based vs middleware-based
- ✓ Pratik: Tam login + protected dashboard

5.2 Sıkça Sorulan Sorular

SSS

S: Tüm component'lere "use client" koysam olmaz mı?

C: Olmaz. JS bundle şişer, SEO ve ilk yükleme yavaşlar. Server component'lerin gücünü kaybedersiniz. Kural: **etkileşim olan yere kadar** server bırakın.

S: localStorage'a token koyabilir miyim?

C: Teknik olarak evet ama XSS saldırılarına karşı savunmasızdır. **httpOnly cookie** her zaman daha güvenlidir. Token'ı sadece sunucu okusun, JavaScript dokunmasın.

S: error.tsx içinde useEffect kullanabilir miyim?

C: Evet. `error.tsx` **her zaman** client component'tir ("use client" zorunlu). Tüm React hook'larını kullanabilirsiniz.

S: Middleware mi layout-based protection mu?

C: Tek bir korumalı bölge varsa **layout-based** yeterli ve okunaklı. Birden fazla farklı korumalı bölge varsa veya logic karmaşıksa **middleware** daha temiz olur. İkisi birlikte de kullanılabilir.

S: Server action güvenli mi?

C: Evet. "use server" fonksiyonları **her zaman sunucuda** çalışır. Client onları sadece çağırabilir, kodunu göremez. Yine de parametreleri **doğrulamayı** unutmayın (input sanitization).

5.3 Sonraki Session — Session 8: Starter Boilerplate

→ Session 8 (14 Mayıs Perşembe)

Tüm bilgileri ekibin gerçek starter projesiyle birleştiriyoruz:

- **Proje yapısı:** folder structure, naming convention
- **Authentication flow:** starter'daki gerçek implementasyon
- **API çağrı yapısı:** services / api client
- **UI library:** @gib-ui/core kullanımı
- **Pratik:** Starter ile sıfırdan yeni feature ekleme